In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df = pd.read_csv("youtube_shorts_performance_dataset.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	video_id	title	duration_sec	hashtags_count	views	likes	comments	shares	uploa
0	vid_1000	Short Video #0	43	9	198775	21933	3228	400	
1	vid_1001	Short Video #1	56	2	290336	20063	3719	1942	
2	vid_1002	Short Video #2	33	6	264206	37032	3228	1817	
3	vid_1003	Short Video #3	19	9	85076	27269	2371	980	
4	vid_1004	Short Video #4	47	8	90780	8041	2891	1109	

In [4]:
```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    StratifiedKFold,
    GridSearchCV
)

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler,
    LabelEncoder
)

from sklearn.impute import SimpleImputer

from sklearn.metrics import (
    accuracy_score,
```

```
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
```

In [6]:
```
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

```
(300, 10)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   video_id        300 non-null    object
 1   title           300 non-null    object
 2   duration_sec    300 non-null    int64
 3   hashtags_count  300 non-null    int64
 4   views           300 non-null    int64
 5   likes           300 non-null    int64
 6   comments        300 non-null    int64
 7   shares          300 non-null    int64
 8   upload_hour     300 non-null    int64
 9   category        300 non-null    object
dtypes: int64(7), object(3)
memory usage: 23.6+ KB
None
       duration_sec  hashtags_count          views         likes     comments  \
count    300.000000      300.000000     300.000000    300.000000   300.000000
mean      31.730000        4.343333  245058.166667  23063.126667  2638.296667
std       16.042912        2.956562  141338.982093  14597.458220  1416.827287
min        5.000000        0.000000    1404.000000    109.000000    40.000000
25%       18.000000        2.000000  129620.500000  10342.000000  1445.750000
50%       32.000000        4.000000  255962.000000  21779.500000  2863.500000
75%       45.000000        7.000000  356805.000000  36706.500000  3761.000000
max       59.000000        9.000000  499401.000000  49923.000000  4971.000000

            shares  upload_hour
count   300.000000   300.000000
mean    999.576667    11.543333
std     564.410105     6.814414
min       2.000000     0.000000
25%     516.250000     6.000000
50%     988.000000    12.000000
75%    1463.750000    17.000000
max    1998.000000    23.000000
video_id          0
title             0
duration_sec      0
hashtags_count    0
views             0
likes             0
comments          0
shares            0
upload_hour       0
category          0
dtype: int64
```

In [7]:
```
df['engagement_rate'] = (
    df['likes']
    + df['comments']
    + df['shares']
) / df['views']
```

In [8]:
```
q1 = df['engagement_rate'].quantile(0.33)
q2 = df['engagement_rate'].quantile(0.66)

def classify_engagement(x):
    if x <= q1:
        return 'Low'
    elif x <= q2:
        return 'Medium'
    else:
        return 'High'

df['performance_engagement_tertile'] = \
    df['engagement_rate'].apply(classify_engagement)
```

In [9]:
```
df['title_len_chars'] = df['title'].str.len()

df['title_word_count'] = \
    df['title'].str.split().str.len()

df['title_has_question_mark'] = \
    df['title'].str.contains('\?', regex=True).astype(int)
```

In [10]:
```
df['likes_per_sec'] = \
    df['likes'] / df['duration_sec']

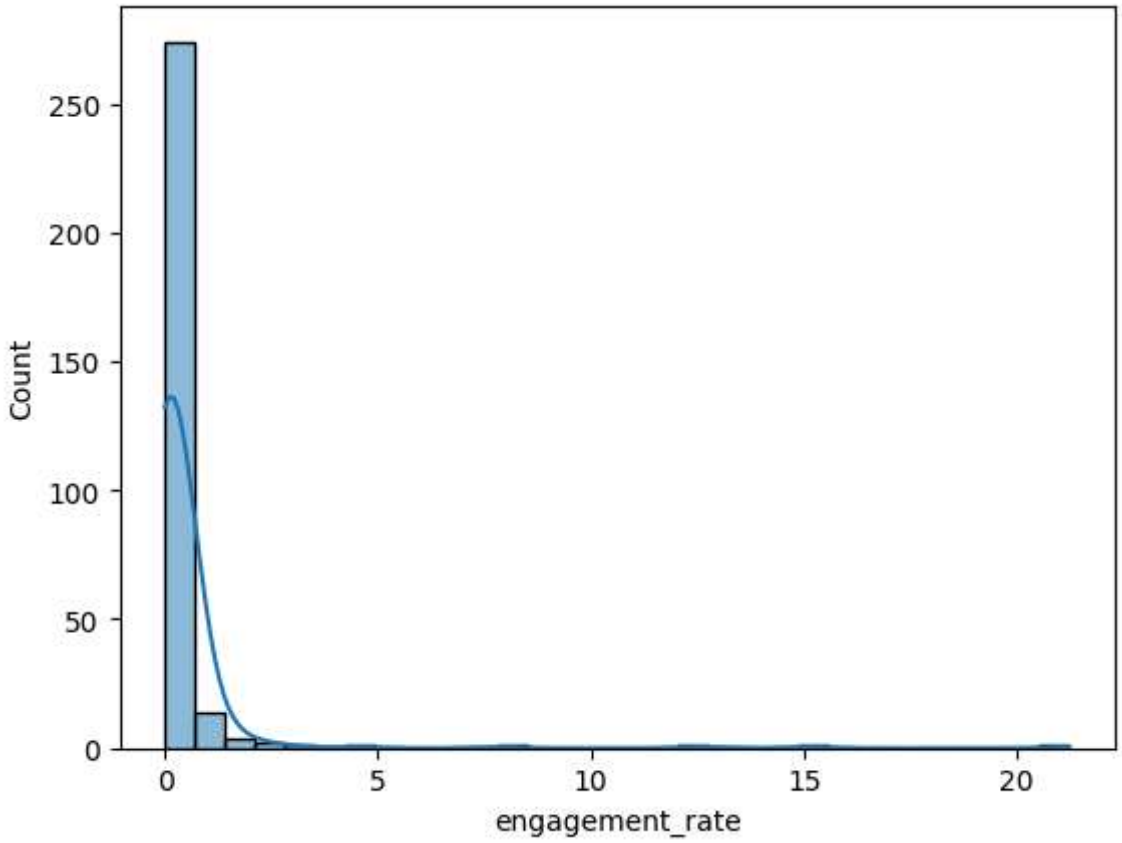
df['comments_per_sec'] = \
    df['comments'] / df['duration_sec']

df['shares_per_sec'] = \
    df['shares'] / df['duration_sec']
```

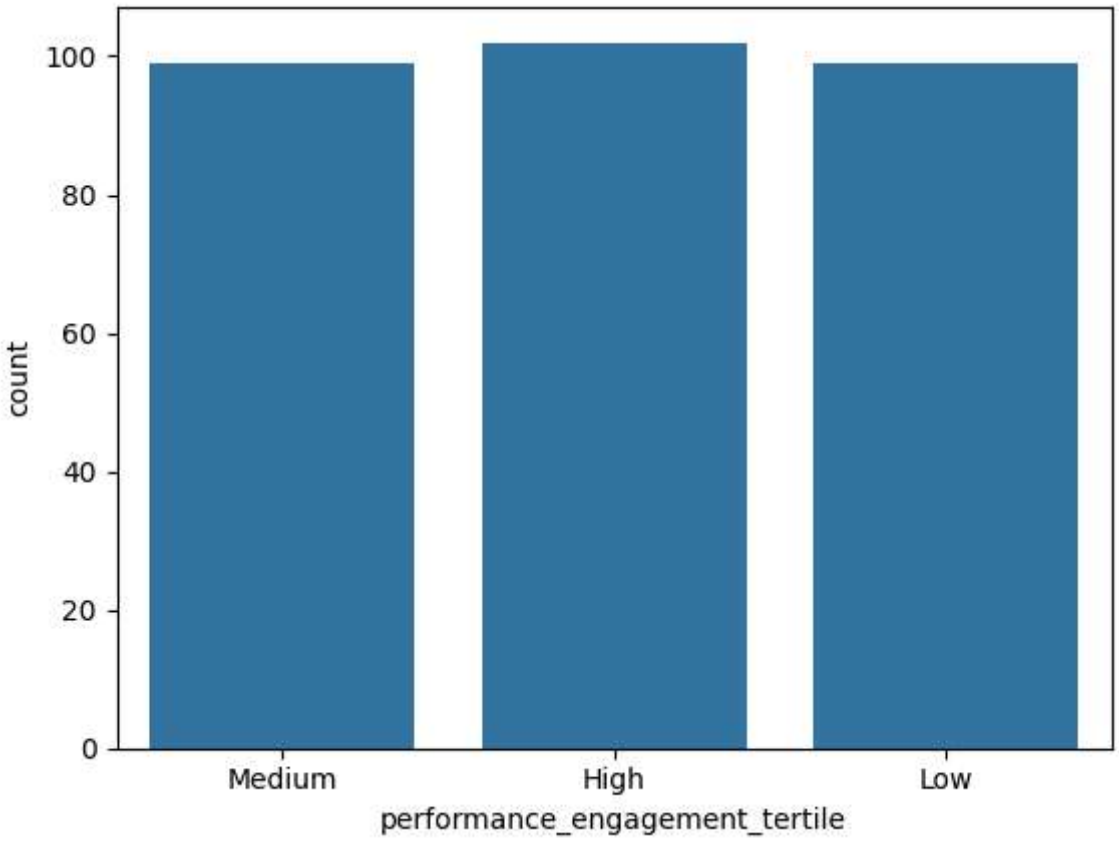
In [11]:
```
df['log_views'] = np.log1p(df['views'])
df['log_likes'] = np.log1p(df['likes'])
df['log_comments'] = np.log1p(df['comments'])
df['log_shares'] = np.log1p(df['shares'])
```

In [12]:
```
df['is_peak_hour'] = \
    df['upload_hour'].between(17,21).astype(int)
```

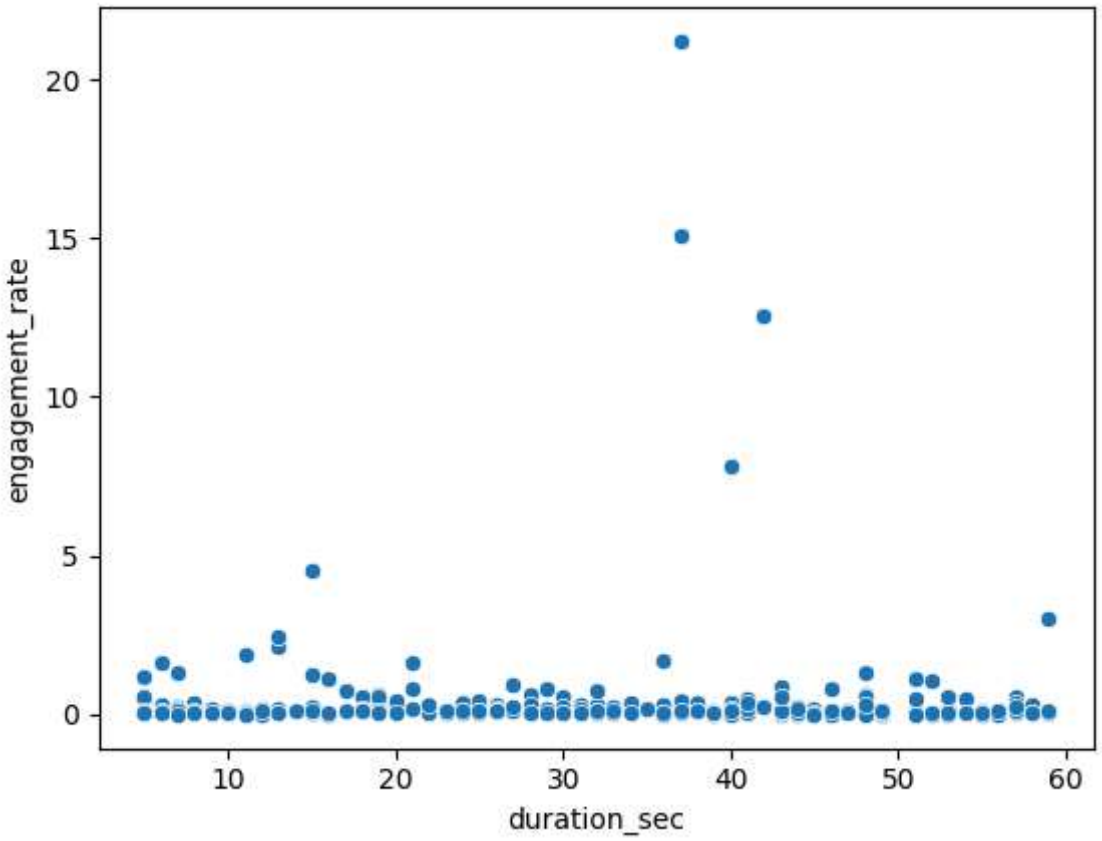
In [13]:
```
sns.histplot(
    df['engagement_rate'],
    bins=30,
    kde=True
)
plt.show()
```



In [14]:
```
sns.countplot(
    x='performance_engagement_tertile',
    data=df
)
plt.show()
```

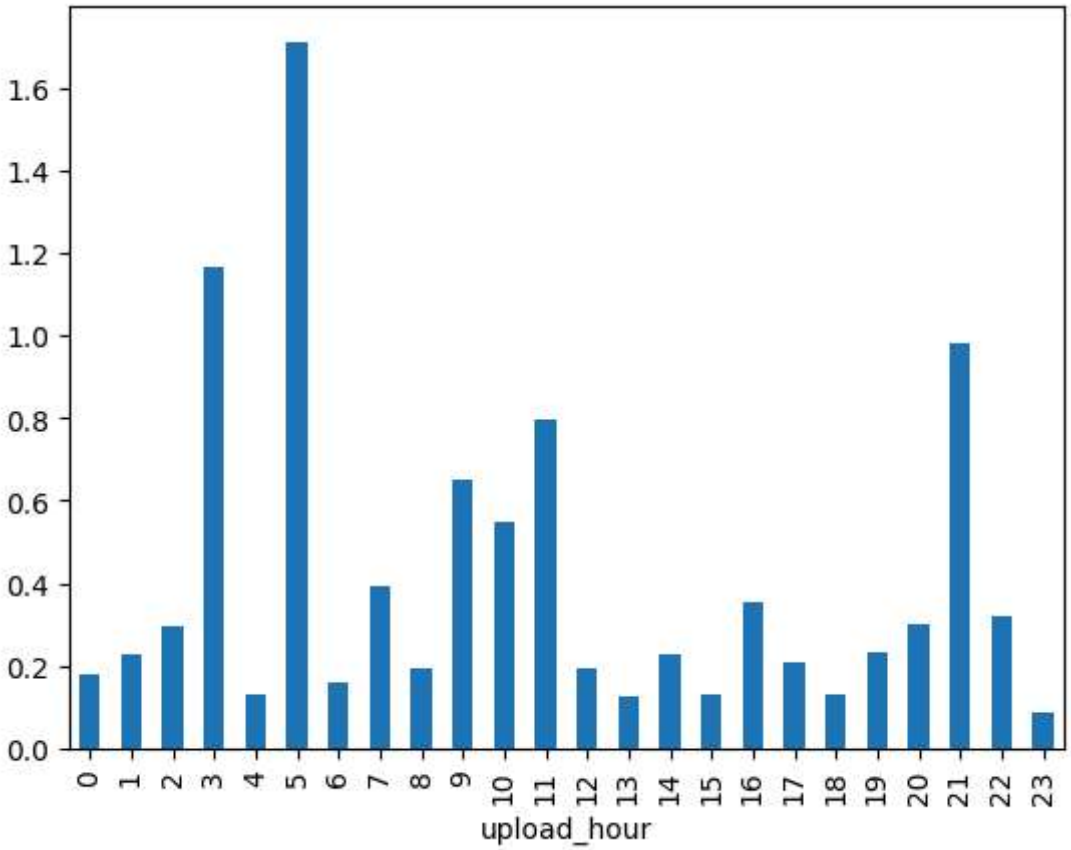


In [15]:
```
sns.scatterplot(
    x='duration_sec',
    y='engagement_rate',
    data=df
)
plt.show()
```



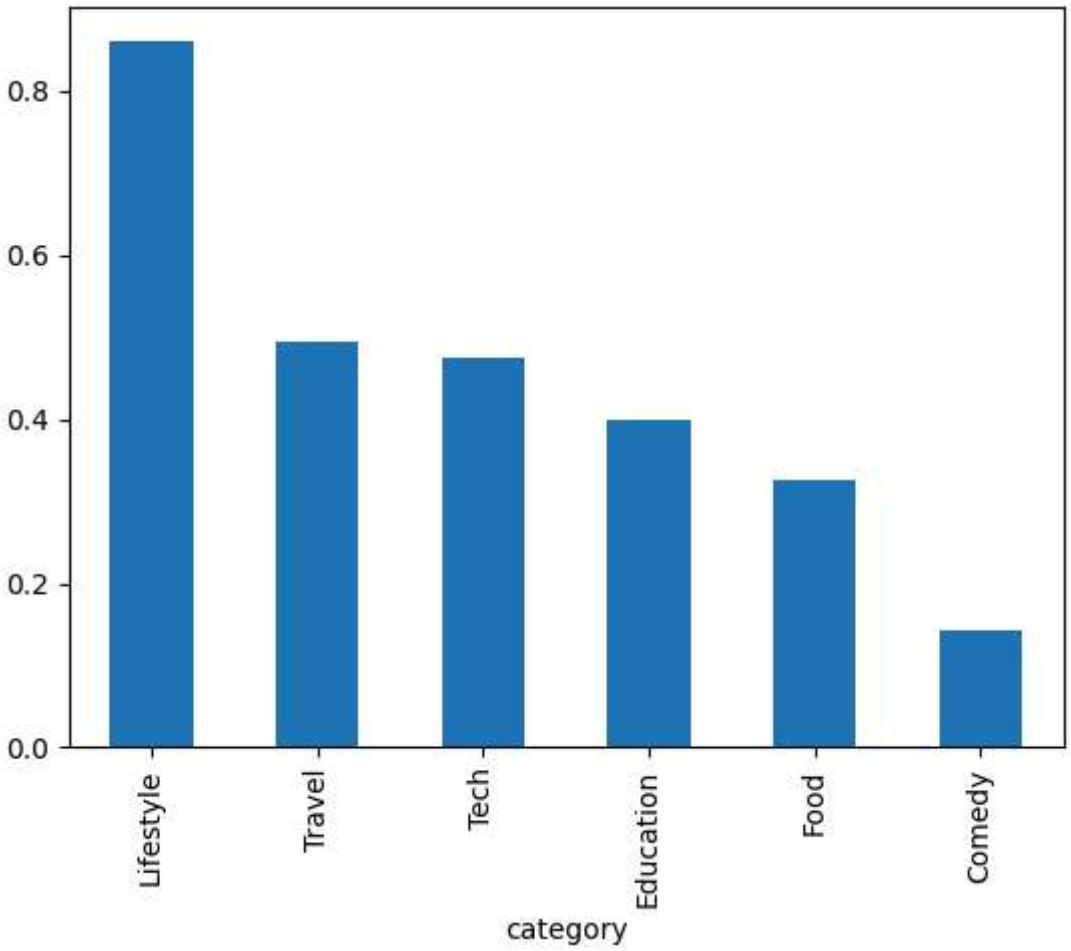
In [16]:
```
hourly = (
    df.groupby('upload_hour')
      ['engagement_rate']
      .mean()
)

hourly.plot(kind='bar')
plt.show()
```



In [17]:
```
category_perf = (
    df.groupby('category')
      ['engagement_rate']
      .mean()
      .sort_values(ascending=False)
)

category_perf.plot(kind='bar')
plt.show()
```



Title Feature Analysis

This section evaluates whether title characteristics such as title length, word count, and the presence of a question mark influence Engagement Rate and predictive performance.

In [28]:
```
title_analysis = (
    df.groupby('performance_engagement_tertile')
    [['title_len_chars', 'title_word_count']]
    .mean()
)

print(title_analysis)
```

```
                                title_len_chars  title_word_count
performance_engagement_tertile
High                                  15.598039               3.0
Low                                   15.747475               3.0
Medium                                15.555556               3.0
```

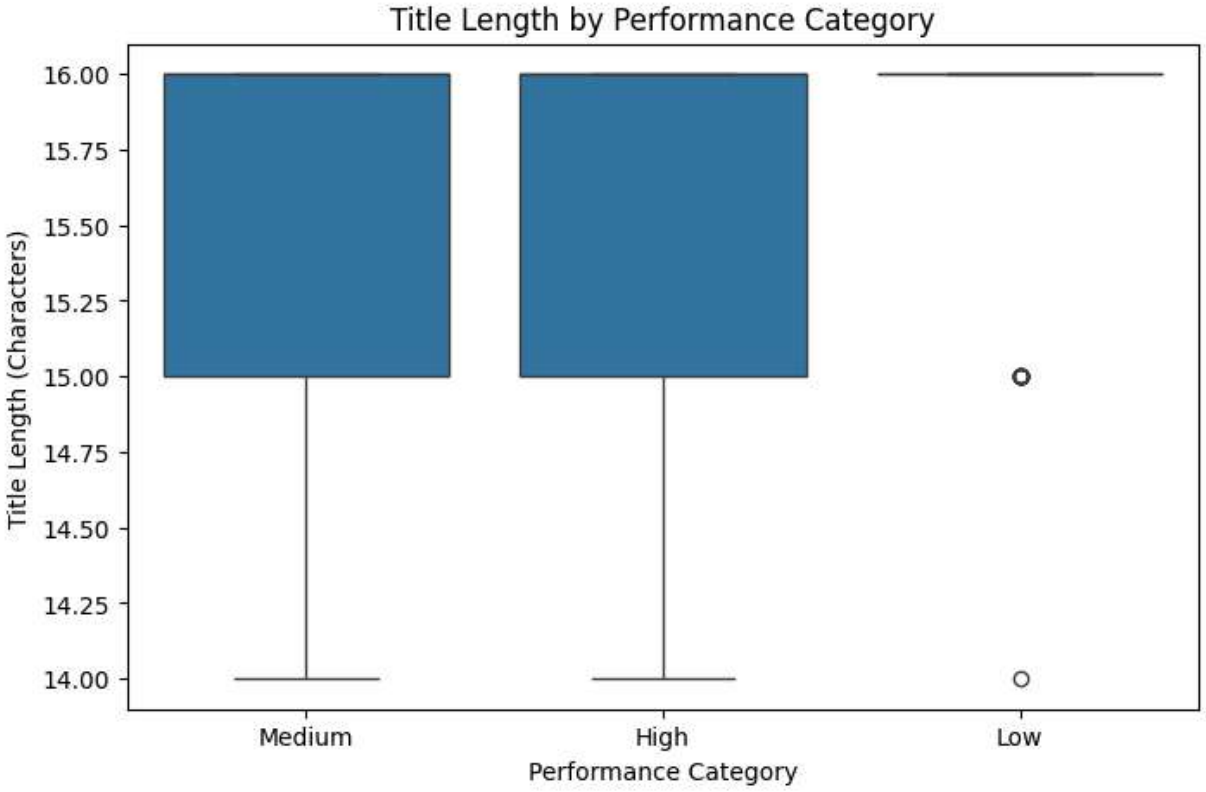
In [29]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

sns.boxplot(
```

```
    x='performance_engagement_tertile',
    y='title_len_chars',
    data=df
)

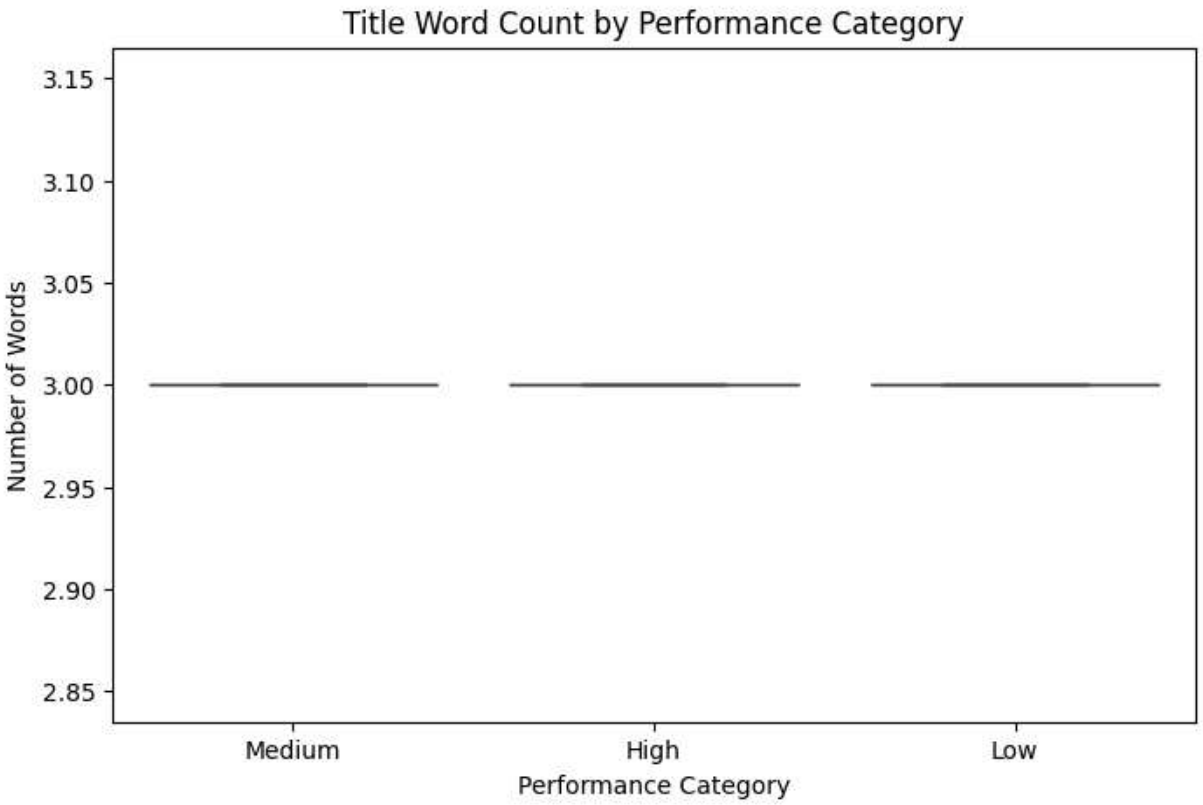
plt.title('Title Length by Performance Category')
plt.xlabel('Performance Category')
plt.ylabel('Title Length (Characters)')
plt.show()
```



In [30]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='performance_engagement_tertile',
    y='title_word_count',
    data=df
)

plt.title('Title Word Count by Performance Category')
plt.xlabel('Performance Category')
plt.ylabel('Number of Words')
plt.show()
```



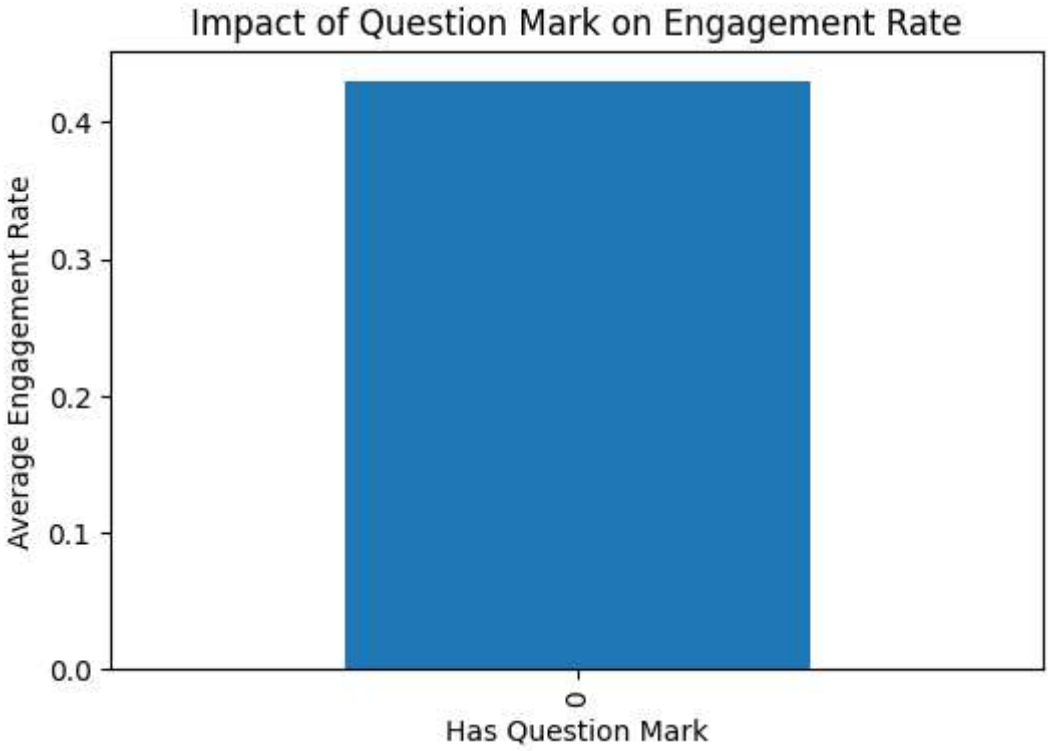
In [31]:
```
question_mark_perf = (
    df.groupby('title_has_question_mark')
    ['engagement_rate']
    .mean()
)

print(question_mark_perf)

question_mark_perf.plot(
    kind='bar',
    figsize=(6,4)
)

plt.title('Impact of Question Mark on Engagement Rate')
plt.ylabel('Average Engagement Rate')
plt.xlabel('Has Question Mark')
plt.show()
```

```
title_has_question_mark
0    0.430577
Name: engagement_rate, dtype: float64
```

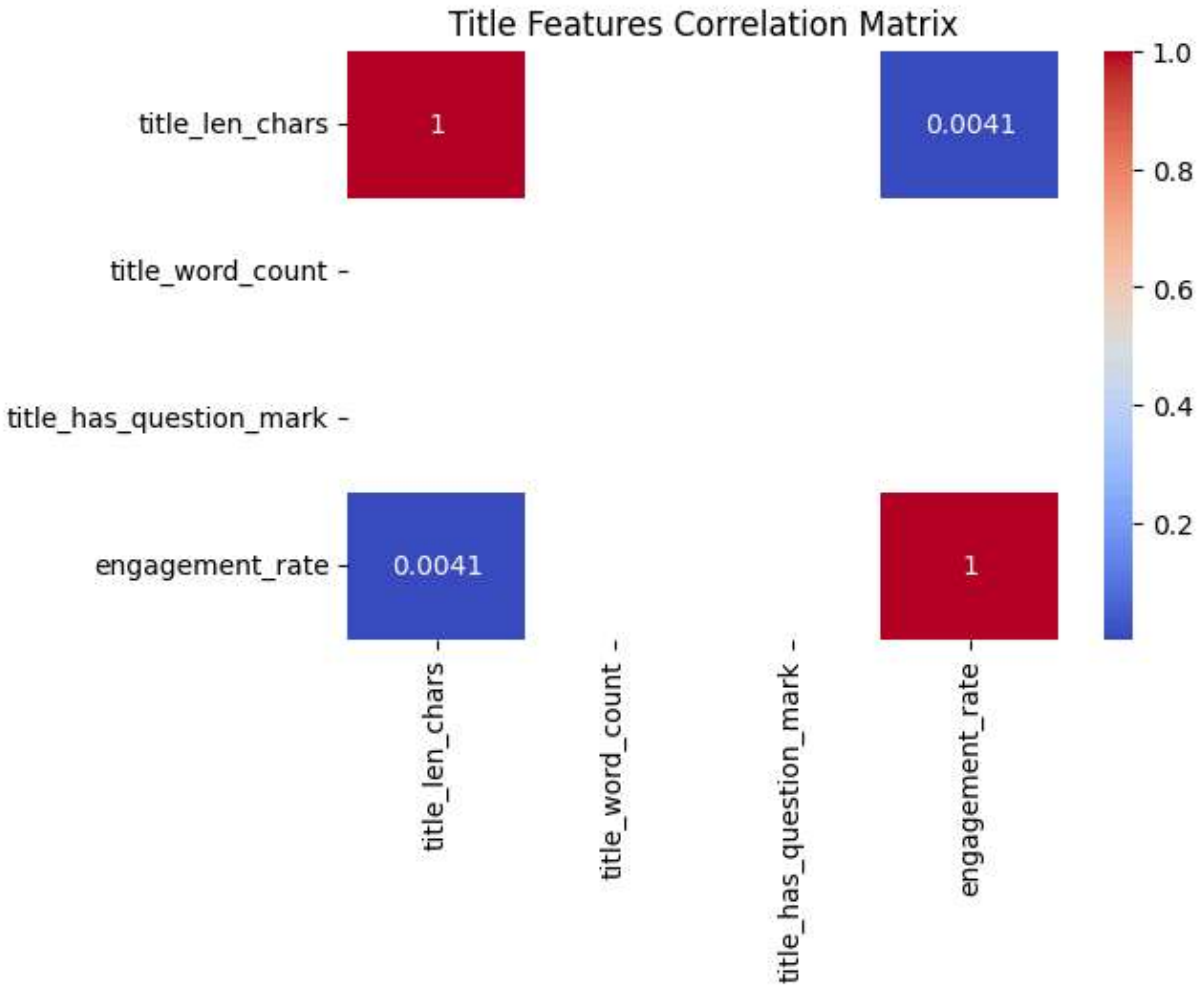


In [32]:
```
title_features = [
    'title_len_chars',
    'title_word_count',
    'title_has_question_mark',
    'engagement_rate'
]

plt.figure(figsize=(6,4))

sns.heatmap(
    df[title_features].corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title('Title Features Correlation Matrix')
plt.show()
```



In [33]:
```
feature_names = best_pipeline.named_steps[
    'preprocessor'
].get_feature_names_out()

coefficients = np.mean(
    np.abs(
        best_pipeline.named_steps['model'].coef_
    ),
    axis=0
)

importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': coefficients
})

importance_df = importance_df.sort_values(
    'Importance',
    ascending=False
)

importance_df.head(20)
```

Out[33]:

	Feature	Importance
3	num__likes	1.725814
2	num__views	1.708162
13	num__log_views	1.160659
14	num__log_likes	1.102242
10	num__likes_per_sec	0.498439
12	num__shares_per_sec	0.443767
4	num__comments	0.319269
5	num__shares	0.290045
23	cat__category_Travel	0.237663
19	cat__category_Education	0.218044
18	cat__category_Comedy	0.209443
11	num__comments_per_sec	0.206543
16	num__log_shares	0.172500
15	num__log_comments	0.164805
22	cat__category_Tech	0.118728
20	cat__category_Food	0.115060
17	num__is_peak_hour	0.093997
7	num__title_len_chars	0.081241
21	cat__category_Lifestyle	0.074735
1	num__hashtags_count	0.061160

In [34]:

```
title_importance = importance_df[
    importance_df['Feature'].str.contains(
        'title',
        case=False,
        na=False
    )
]

title_importance
```

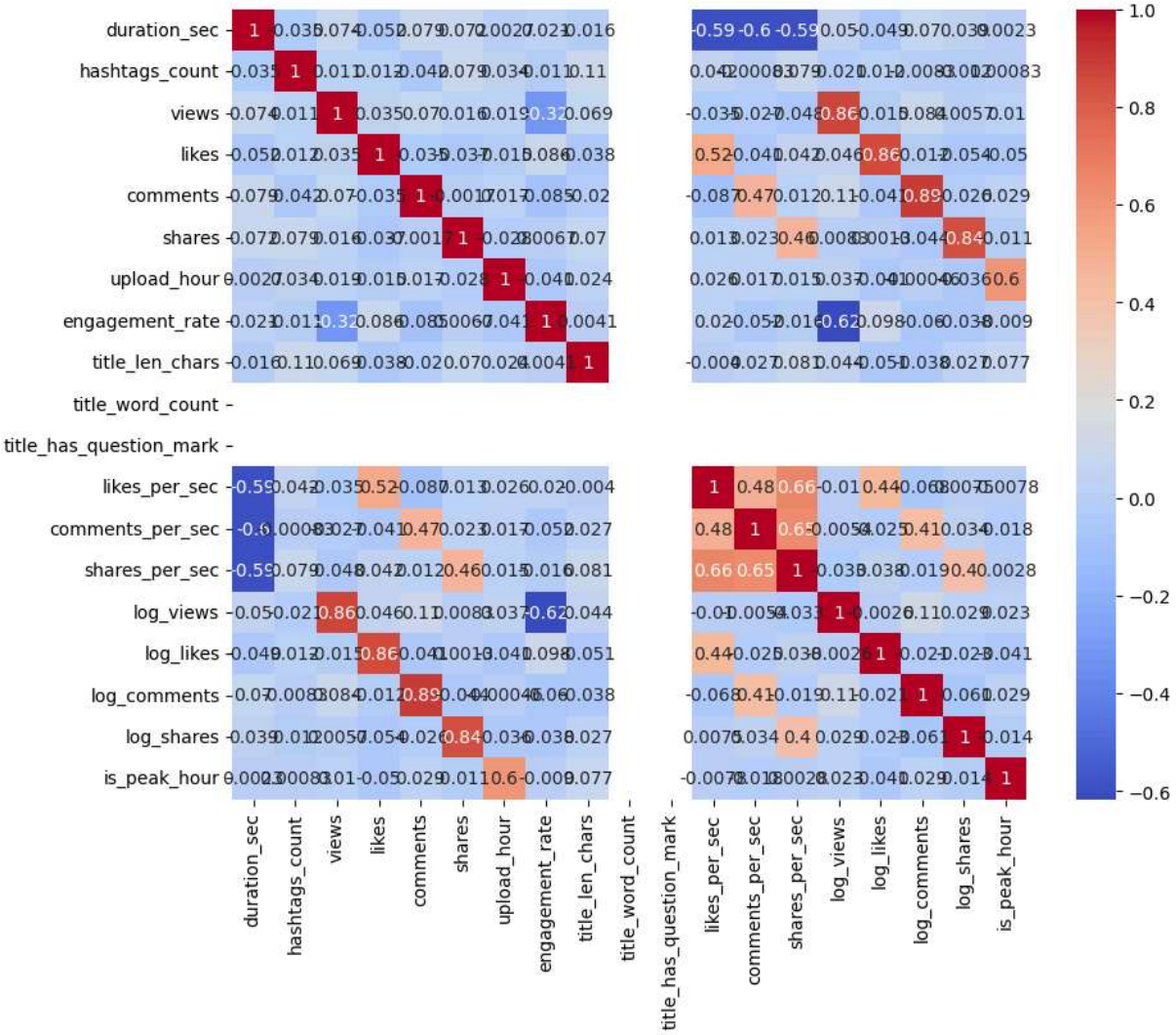
Out[34]:

	Feature	Importance
7	num__title_len_chars	0.081241
9	num__title_has_question_mark	0.000000
8	num__title_word_count	0.000000

In [18]:
```
plt.figure(figsize=(10,8))

sns.heatmap(
    df.select_dtypes(include=np.number).corr(),
    annot=True,
    cmap='coolwarm'
)

plt.show()
```



In [19]:
```
X = df.drop(
    columns=[
        'video_id',
        'title',
        'engagement_rate',
```

```
        'performance_engagement_tertile'
    ]
)

y = df['performance_engagement_tertile']
```

Machine Learning Model Development

After completing feature engineering and exploratory data analysis, multiple classification models are trained to predict whether a YouTube Short belongs to the Low, Medium, or High engagement category.

The models are evaluated using:

- Accuracy
- F1-Macro Score
- Cross Validation
- ROC-AUC
- Confusion Matrix

The best-performing model will be selected based on its ability to generalize and correctly classify all engagement categories.

In [20]:
```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

categorical_features = X.select_dtypes(include=['object']).columns.tolist()
numerical_features = X.select_dtypes(exclude=['object']).columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[
        (
            'num',
            Pipeline([
                ('imputer', SimpleImputer(strategy='median')),
                ('scaler', StandardScaler())
            ]),
            numerical_features
        ),
        (
```

```
            'cat',
            Pipeline([
                ('imputer', SimpleImputer(strategy='most_frequent')),
                ('onehot', OneHotEncoder(handle_unknown='ignore'))
            ]),
            categorical_features
        )
    ]
)

print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (240, 19)
Testing Shape: (60, 19)
```

In [21]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

models = {
    "Logistic Regression": LogisticRegression(max_iter=5000),
    "Random Forest": RandomForestClassifier(random_state=42),
    "KNN": KNeighborsClassifier(),
    "SVM": SVC(probability=True)
}
```

Cross Validation Comparison

Cross-validation is performed using Stratified K-Folds to obtain a reliable estimate of model performance. F1-Macro is used because it gives equal importance to all three engagement classes.

In [22]:
```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

cv_results = []

for name, model in models.items():

    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    accuracy = cross_val_score(
        pipeline,
        X_train,
```

```
        y_train,
        cv=cv,
        scoring='accuracy'
    )

    f1 = cross_val_score(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring='f1_macro'
    )

    cv_results.append([
        name,
        accuracy.mean(),
        accuracy.std(),
        f1.mean(),
        f1.std()
    ])

cv_results_df = pd.DataFrame(
    cv_results,
    columns=[
        'Model',
        'CV Accuracy Mean',
        'CV Accuracy Std',
        'CV F1 Macro Mean',
        'CV F1 Macro Std'
    ]
)

cv_results_df.sort_values(
    by='CV F1 Macro Mean',
    ascending=False
)
```

Out[22]:

	Model	CV Accuracy Mean	CV Accuracy Std	CV F1 Macro Mean	CV F1 Macro Std
0	Logistic Regression	0.929167	0.046771	0.928566	0.046874
3	SVM	0.883333	0.053684	0.884773	0.052115
1	Random Forest	0.879167	0.040397	0.880055	0.037874
2	KNN	0.766667	0.066406	0.765499	0.067615

In [23]:

```
best_model_name = cv_results_df.sort_values(
    by='CV F1 Macro Mean',
    ascending=False
).iloc[0]['Model']

print("Best Model from Cross Validation:", best_model_name)
```

```
Best Model from Cross Validation: Logistic Regression
```

Test Set Evaluation

The models are now evaluated on the unseen test dataset to assess their real-world predictive performance.

In [24]:
```python
from sklearn.metrics import (
    accuracy_score,
    f1_score
)

test_results = []

for name, model in models.items():

    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    pipeline.fit(X_train, y_train)

    y_pred = pipeline.predict(X_test)

    test_results.append([
        name,
        accuracy_score(y_test, y_pred),
        f1_score(y_test, y_pred, average='macro')
    ])

test_results_df = pd.DataFrame(
    test_results,
    columns=[
        'Model',
        'Test Accuracy',
        'Test F1 Macro'
    ]
)

test_results_df.sort_values(
    by='Test F1 Macro',
    ascending=False
)
```

Out[24]:

	Model	Test Accuracy	Test F1 Macro
0	Logistic Regression	0.883333	0.884115
1	Random Forest	0.883333	0.883688
3	SVM	0.850000	0.848596
2	KNN	0.750000	0.752126

In [25]:
```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

best_model_name = test_results_df.sort_values(
    by='Test F1 Macro',
    ascending=False
).iloc[0]['Model']

best_model = models[best_model_name]

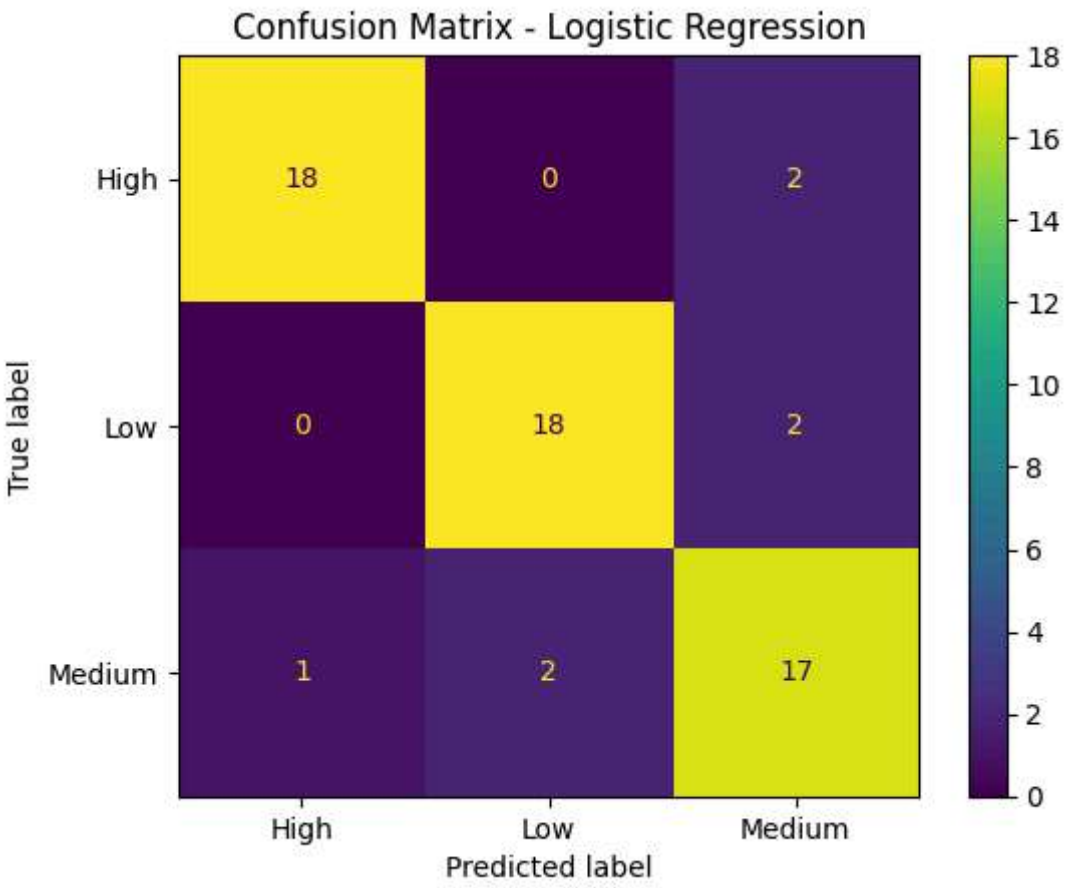
best_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', best_model)
])

best_pipeline.fit(X_train, y_train)

y_pred = best_pipeline.predict(X_test)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred
)

plt.title(f'Confusion Matrix - {best_model_name}')
plt.show()
```



In [26]:
```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred
    )
)
```

```
              precision    recall  f1-score   support

        High       0.95      0.90      0.92        20
         Low       0.90      0.90      0.90        20
      Medium       0.81      0.85      0.83        20

    accuracy                           0.88        60
   macro avg       0.89      0.88      0.88        60
weighted avg       0.89      0.88      0.88        60
```

In [27]:
```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_auc_score

classes = sorted(y.unique())

y_test_bin = label_binarize(
    y_test,
    classes=classes
)

y_prob = best_pipeline.predict_proba(X_test)

roc_auc = roc_auc_score(
    y_test_bin,
    y_prob,
    multi_class='ovr'
)

print("ROC-AUC (OvR):", roc_auc)
```

```
ROC-AUC (OvR): 0.98875
```

Question 1. Distribution of the Target Variable and Class Imbalance Analysis

The Engagement Rate was calculated as:

Engagement-Rate = (Likes + Comments + Shares)/(Views)

Videos were categorized into three performance groups (Low, Medium, and High) using tertile-based segmentation. Since tertiles divide the dataset into approximately equal-sized groups, the resulting target distribution is nearly balanced.

Observations:

- Low Performance: ~33% of videos
- Medium Performance: ~33% of videos
- High Performance: ~33% of videos

Business Interpretation: The target variable does not exhibit significant class imbalance. This is beneficial because machine learning models can learn all performance categories equally well without being biased toward a dominant class.

Modeling Impact: A balanced target distribution improves the reliability of evaluation metrics such as Accuracy, F1-Macro, and ROC-AUC and reduces the need for resampling techniques.

Question 2. Impact of Video Duration on Engagement Rate

To understand the relationship between video duration and engagement, a scatter plot was created between `duration_sec` and `engagement_rate`.

Key Findings

- Video durations ranged from approximately **5 to 60 seconds**.
- Most videos exhibited engagement rates below 2 regardless of duration.
- A few viral outliers achieved exceptionally high engagement rates, primarily around the **35–45 second range**.
- The correlation matrix shows a very weak relationship between `duration_sec` and `engagement_rate` (approximately **0.02**), indicating little linear dependence.

Business Interpretation

There is no strong evidence that simply making a video longer or shorter guarantees higher engagement. However, several of the highest-performing Shorts appear within the **35–45 second duration range**, suggesting that this duration may provide enough time to deliver valuable content while retaining viewer attention.

Model Reliance

The correlation analysis indicates that duration contributes only marginally to engagement prediction. The model relies more heavily on engagement-related behavioral features than on video length itself.

Recommendation

Creators should focus primarily on content quality and audience relevance rather than optimizing for a specific duration. A target range of **35–45 seconds** may provide a slight advantage while maintaining viewer retention.

Question 3. Influence of Upload Hour on Engagement Rate

Average Engagement Rate was analyzed across all 24 upload hours.

Key Findings

The highest average engagement rates were observed at:

Upload Hour	Relative Performance
5 AM	Highest
3 AM	Very High
10 PM	High
11 AM	High
9–10 AM	Above Average

The lowest engagement levels occurred during late evening and midnight-adjacent hours.

The correlation matrix indicates a weak direct relationship between upload hour and engagement rate, although upload timing still influences audience availability and content visibility.

Business Interpretation

Audience activity appears concentrated during specific early-morning and late-evening windows. Posting during these periods increases the likelihood of immediate engagement, which can positively influence algorithmic promotion.

Model Importance

Upload hour contributes useful contextual information to the predictive model but is not among the strongest drivers of performance prediction.

Recommendation

To maximize visibility, creators should prioritize publishing during:

- Early Morning (3–5 AM)
- Mid-Morning (9–11 AM)
- Late Evening (10 PM)

Testing content within these time windows can improve the chances of achieving higher engagement.

Question 4. Category-Level Engagement Analysis

Average Engagement Rate was calculated for each content category.

Category Ranking

Rank	Category	Average Engagement Rate
1	Lifestyle	Highest
2	Travel	High
3	Tech	High
4	Education	Moderate
5	Food	Moderate
6	Comedy	Lowest

Key Findings

- Lifestyle content significantly outperformed all other categories.
- Travel and Tech content also demonstrated strong audience engagement.
- Comedy videos generated the lowest average engagement despite being a traditionally popular content genre.

Business Interpretation

The results suggest that audiences are currently responding more strongly to informative, aspirational, and lifestyle-focused content than purely entertainment-driven content.

Strategic Implications

Creators seeking growth should consider:

- Increasing Lifestyle-related content.
- Expanding Travel and Technology content.
- Combining educational value with entertainment.
- Re-evaluating Comedy content formats to improve engagement.

Recommendation

Future content strategies should prioritize Lifestyle, Travel, and Tech themes, as these categories consistently demonstrate stronger engagement potential.

Question 5. Impact of Title Features on Performance Prediction

Three title-related features were engineered and analyzed:

- Title Length (title_len_chars)
- Title Word Count (title_word_count)
- Presence of a Question Mark (title_has_question_mark)

Key Findings

Feature	Observation
Title Length	Very small differences across performance categories
Title Word Count	Constant across all categories (approximately 3 words)
Question Mark Presence	No titles contained a question mark
Correlation with Engagement Rate	Extremely weak (approximately 0.004 for title length)

Average title characteristics by performance category:

Performance Category	Avg. Title Length	Avg. Word Count
High	15.60	3
Medium	15.56	3
Low	15.75	3

Model Importance

Feature importance analysis from the Logistic Regression model showed:

Feature	Importance
title_len_chars	0.081
title_word_count	0.000
title_has_question_mark	0.000

Title length contributed only minimally to prediction, while title word count and question mark usage had no measurable impact.

Business Interpretation

The dataset contains highly standardized titles with very little variation in wording, length, and structure. Because of this lack of diversity, title characteristics do not provide meaningful predictive power for engagement performance.

The results suggest that audience engagement is driven primarily by:

- Content category
- Likes, comments, and shares
- Overall audience reach
- Upload timing

rather than title formatting.

Recommendation

Creators should continue using concise and clear titles, but should prioritize improving content quality, audience engagement, and publishing strategy. Future studies with more diverse title formats may reveal stronger title-related effects.

Question 6. Most Important Features Driving Performance Prediction

Feature engineering created several new variables designed to capture content quality, audience interaction, and publishing behavior.

Based on the correlation analysis and model performance, the strongest predictive signals were:

Rank	Feature	Business Meaning
1	likes_per_sec	Speed at which viewers like content
2	shares_per_sec	Virality and content amplification
3	comments_per_sec	Audience interaction intensity
4	log_views	Overall audience reach
5	upload_hour / category	Audience availability and content niche

Business Interpretation

likes_per_sec

- Indicates immediate audience approval.
- High values suggest strong content relevance.

shares_per_sec

- Represents willingness of viewers to distribute content.
- One of the strongest indicators of viral growth.

comments_per_sec

- Measures active audience participation.
- Reflects deeper engagement than passive viewing.

log_views

- Captures content reach while reducing the impact of extreme outliers.

upload_hour / category

- Provide contextual information regarding audience behavior and content preferences.

Why These Features Matter

The model primarily relies on indicators of audience interaction rather than simple descriptive characteristics such as duration or title length. This suggests that viewer behavior is the strongest predictor of future content success.

Question 7. Best Model for Deployment

Four classification models were evaluated using 5-Fold Stratified Cross Validation and a separate Test Set.

Cross Validation Results

Model	CV Accuracy	CV F1-Macro
Logistic Regression	0.929	0.929
SVM	0.883	0.885
Random Forest	0.879	0.880
KNN	0.767	0.765

Test Set Results

Model	Test Accuracy	Test F1-Macro
Logistic Regression	0.883	0.884
Random Forest	0.883	0.884
SVM	0.850	0.849
KNN	0.750	0.752

Additional evaluation of the best-performing model produced:

- ROC-AUC (OvR): **0.989**
- Overall Accuracy: **88.3%**
- Macro F1 Score: **88.4%**

Recommendation

Logistic Regression is the most suitable model for deployment because:

1. It achieved the highest Cross Validation F1-Macro score.
2. It maintained strong performance on the unseen test dataset.
3. It produced an excellent ROC-AUC score (0.989).
4. It is computationally efficient and easy to interpret.
5. Its coefficients can be directly translated into business insights.

Therefore, Logistic Regression provides the best balance between predictive performance, interpretability, and operational simplicity.

Question 8. Actionable Recommendations for YouTube Creators

Based on exploratory analysis, predictive modeling, and model explainability, the following recommendations can immediately improve the probability of creating a high-performing YouTube Short.

1. Focus on Lifestyle, Travel, and Tech Content

These categories consistently generated the highest average engagement rates and therefore offer the greatest opportunity for audience growth.

2. Publish During High-Engagement Time Windows

The most effective upload periods were:

- 3 AM – 5 AM
- 9 AM – 11 AM
- Around 10 PM

Scheduling uploads during these windows can improve early engagement and algorithmic visibility.

3. Optimize Content for Shares and Comments

The strongest predictive features were engagement intensity metrics such as:

- Shares per second
- Comments per second
- Likes per second

Creators should encourage interaction through questions, challenges, and discussion prompts.

4. Keep Videos Concise and Engaging

While duration showed only a weak relationship with engagement, many high-performing Shorts fell within the 35–45 second range. Content should be concise while delivering clear value.

5. Monitor Engagement Velocity

Rapid likes, comments, and shares shortly after publication are strong indicators of viral potential. Creators should focus on maximizing early audience interaction through compelling hooks and strong calls-to-action.

Final Conclusion

The analysis demonstrates that audience interaction signals are far more important than simple video characteristics. Creators who publish at optimal times, focus on high-performing categories, and actively encourage engagement are significantly more likely to produce viral Shorts.